Grammar Fundamentals (Fast Expression)

This article explains the **grammar fundamentals** of the BRAIN Fast Expression language. The objective is to help readers **read and write expressions correctly from a syntactic perspective**, without requiring deep quantitative finance or programming expertise.

Fast Expression is not a general-purpose programming language. It is a structured expression language designed to clearly communicate financial modeling logic. As a result, understanding its grammar is more important than mastering complex formulas.

---

1. What Kind of Language Is Fast Expression?

Fast Expression is a proprietary pseudo code language used on the BRAIN platform to define and test Alphas. Its key characteristics are:

- It is not an object-oriented language (no classes, objects, or pointers)
- It does not include loops or traditional control flow
- All logic is expressed through nested expressions

Fast Expression is designed to **prioritize readability of logic over implementation detail**.

---

2. Sentence Structure in Fast Expression

Fast Expression can be directly compared to natural language grammar:

- Data fields → nouns (the data itself)
- Operators → verbs (actions applied to data)
- Numerical values → modifiers (parameters such as window length)

The fundamental structure of every expression is:

```
operator(argument1, argument2, ...)
```

Every operator must use parentheses, and all arguments must be separated by commas.

---

3. Data Fields (The Raw Materials)

A data field is a named collection of data, such as prices, volumes, or returns. Conceptually, each data field is a matrix indexed by **assets × time**.

Common examples include: - close - open - volume - returns

A critical grammar rule is that **data fields can stand alone**, but they have no strategic meaning until processed by an operator.

Examples: - `close` → raw price data - `rank(close)` → a signal derived from prices

---

4. Operators (The Actions)

Operators are mathematical transformations applied to data fields to generate signals. While operators come in many categories, grammar-wise they are defined by how they are written and parameterized.

Common operator types: - Arithmetic: +, -, *, /, ^ - Transformational: log(x) - Cross-sectional: rank(x) - Time-series: ts_rank(x, n)

Important grammar rules: - Time-series operators always require a window parameter - Cross-sectional operators do not reference historical time windows

---

5. Nesting Expressions

Fast Expression allows unlimited nesting of operators. The output of an inner expression becomes the input of the outer expression.

Example:

```
rank(ts_rank(returns, 20))
```

Correct reading order: 1. Compute the time-series rank of returns over the past 20 days 2. Rank those values cross-sectionally across assets

Always read expressions **from the inside out**.

---

6. Multiple Expressions and Semicolons

A Fast Expression script may contain multiple expressions separated by semicolons.

Grammar rules: - All lines except the last should end with a semicolon - The final expression is the Alpha used by the simulator

Example:

```
a = ts_rank(returns, 20);
b = rank(a);
b
```

---

7. Comments

Comments are used to explain logic and improve readability. They do not affect computation.

Block comment syntax:

```
/* explanatory text */
```

Comments exist to help humans understand expressions, not to improve Alpha performance.

---

8. Common Grammar Errors

- Omitting the window parameter in time-series operators
- Reading expressions left-to-right instead of inside-out
- Writing multiple expressions but misunderstanding that only the final one is used
- Confusing cross-sectional operators with time-series operators

Fast Expression is not complex, but it is unforgiving with respect to grammar errors.

Summary

Mastering the grammar fundamentals of Fast Expression is the most important first step. Once readers can reliably identify data fields, operators, and parameters, they are well prepared to progress toward more advanced Alpha construction concepts.